



17.6.5. Detect Cycle in Directed Graph

Problem :

- ✓ Given **Directed Graph**.
- ✓ You need to Check whether Cycle is Present or not.
- ✓ **DFS** based Solution

Lab13.java

```
package com.myjlc.graphs.problems;

public class Lab13 {

    public static boolean isCyclePresent(DirectedGraph mygraph) {

        boolean visited[] = new boolean[mygraph.vertexCount];
        boolean callStack[] = new boolean[mygraph.vertexCount];

        for (int i = 0; i < mygraph.vertexCount; i++) {
            if (!visited[i]) {
                if(dfs(mygraph, i, visited,callStack))
                    return true;
            }
        }

        return false;
    }

    public static boolean dfs(DirectedGraph mygraph,int currVertex,boolean
visited[], boolean callStack[]) {

        visited[currVertex] = true;
        callStack[currVertex] = true;
```



```
        for(int vertex: mygraph.adjList.get(currVertex)) {
            if(!visited[vertex]) {
                if(dfs(mygraph, vertex, visited,callStack))
                    return true;
            }else if(callStack[vertex]==true){
                return true;
            }
        }
        callStack[currVertex] = false;
        return false;
    }

    public static void main(String[] args) {

        DirectedGraph mygraph = new DirectedGraph(6);

        mygraph.addEdge(0, 1);
        mygraph.addEdge(2, 1);
        mygraph.addEdge(2, 3);
        mygraph.addEdge(3, 4);
        mygraph.addEdge(4, 5);
        //mygraph.addEdge(5, 3);

        boolean flag = isCyclePresent(mygraph);
        System.out.println(flag);

    }
}
```



17.6.6. Detect Cycle in Directed Graph

Problem :

- ✓ Given **Directed Graph**.
- ✓ You need to Check whether Cycle is Present or not.
- ✓ **Based on Kahn's Algorithm**

Lab14.java

```
package com.myjlc.graphs.problems;

import java.util.LinkedList;
import java.util.Queue;

public class Lab14 {
    public static boolean isCyclePresent(DirectedGraph mygraph) {

        //Step1: Find Indegrees of All the Vertices
        int indegrees[] = new int[mygraph.vertexCount];

        for(int i=0;i<indegrees.length;i++) {
            for(int vertex:mygraph.adjList.get(i)) {
                indegrees[vertex]++;
            }
        }

        //Step2: Create the Queue
        Queue<Integer> myque = new LinkedList<Integer>();

        //Step3: Add Zero Indegree Vertices to Queue
        for(int i=0;i<indegrees.length;i++) {
            if(indegrees[i]==0)
                myque.add(i);
        }

        //Step4: Take the Counter
        int count=0;
```



```
//Step5 : As long as Queue is Not Empty
while(!myque.isEmpty()) {
    //5.A. Get Vertex from Queue
    int currVertex =myque.poll();

    //5B. For Every Adjacent Vertex of currVertex
    for(int vertex:mygraph.adjList.get(currVertex)) {

        //a)Reduce the Indegree of Vertex by 1
        indegrees[vertex]--;

        //b)If Indegree of that vertex becomes zero then
        //add to Queue
        if(indegrees[vertex]==0)
            myque.add(vertex);
    }

    //5C. Increase the Count
    count++;
}
return (mygraph.vertexCount!=count);
}

public static void main(String[] args) {

    DirectedGraph mygraph = new DirectedGraph(5);
    mygraph.addEdge(0, 2);
    mygraph.addEdge(0, 3);
    mygraph.addEdge(1, 3);
    mygraph.addEdge(1, 4);
    mygraph.addEdge(2, 3);

    boolean flag = isCyclePresent(mygraph);
    System.out.println(flag);
}
}
```

17.6.7. Implementing Graph with Weights on Edges

Problem :

- ✓ Implementing Graph with Weights on Edges.

AdjacentNode.java

```
package com.myjlc.graphs.problems;

public class AdjacentNode {
    int vertex;
    int weight;

    public AdjacentNode() {
    }

    public AdjacentNode(int vertex, int weight) {
        super();
        this.vertex = vertex;
        this.weight = weight;
    }

    @Override
    public String toString() {
        return "[vertex=" + vertex + ", weight=" + weight + "]";
    }
}
```



MyDirectedGraph.java

```
package com.myjlc.graphs.problems;

import java.util.ArrayList;
import java.util.LinkedList;

class MyDirectedGraph {

    int vertexCount;
    ArrayList<LinkedList<AdjacentNode>> adjList;

    public MyDirectedGraph(int vertexCount) {

        this.vertexCount = vertexCount;
        this.adjList = new ArrayList<LinkedList<AdjacentNode>>(vertexCount);

        for (int i = 0; i < vertexCount; i++) {
            this.adjList.add(new LinkedList<AdjacentNode>());
        }
    }

    public void addEdge(int vertex1, int vertex2, int weight) {
        AdjacentNode mynode = new AdjacentNode(vertex2, weight);
        adjList.get(vertex1).add(mynode);
    }

    public void printGraph() {
        for (int i = 0; i < vertexCount; i++) {
            System.out.print("Vertex : " + i + "=> \t");
            for (int j = 0; j < adjList.get(i).size(); j++) {
                System.out.print(adjList.get(i).get(j) + "\t");
            }
            System.out.println("");
        }
    }
}
```



Lab15.java

```
package com.myjlc.graphs.problems;

public class Lab15 {

    public static void main(String[] args) {

        MyDirectedGraph mygraph = new MyDirectedGraph(6);

        mygraph.addEdge(0, 1,2);
        mygraph.addEdge(0, 4,1);

        mygraph.addEdge(1, 2,3);
        mygraph.addEdge(4,2,2);

        mygraph.addEdge(2, 3,6);
        mygraph.addEdge(4, 5,4);
        mygraph.addEdge(5, 3,1);

        mygraph.printGraph();

    }
}
```



17.6.8. Topological Ordering of DAG

Problem :

- ✓ Given **DAG(Directed Acyclic Graph)** with **Weights on Edges**
- ✓ You need to find the Topological Ordering/ Sorting of given **DAG**.
- ✓ **BFS based Solution (Kahns Algorithm)**

Lab16.java

```
package com.myjlc.graphs.problems;
```

```
import java.util.ArrayList;  
import java.util.LinkedList;  
import java.util.Queue;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab16 {
```

```
public static ArrayList<Integer> topologicalOrder(MyDirectedGraph  
mygraph) {
```

```
    ArrayList<Integer> topOrder = new  
    ArrayList<Integer>(mygraph.vertexCount);
```

```
//Step1: Find Indegrees of All Vertices
```

```
int indegrees[] = new int[mygraph.vertexCount];
```

```
for(int i=0;i<indegrees.length;i++) {  
    for(AdjacentNode mynode:mygraph.adjList.get(i)) {  
        indegrees[mynode.vertex]++;  
    }  
}
```



//Step2: Create Queue

```
Queue<Integer> myque = new LinkedList<Integer>();
```

//Step3: Add Zero-Indegree Vertices to Queue

```
for(int i=0;i<indegrees.length;i++) {  
if(indegrees[i]==0)  
myque.add(i);  
}
```

//4.When Queue is Not Empty

```
while(!myque.isEmpty()) {
```

//4A. Get Vertex from Queue

```
int currVertex =myque.poll();  
topOrder.add(currVertex);
```

//4B. For each Adjacent Vertex

```
for(AdjacentNode mynode:mygraph.adjList.get(currVertex)) {
```

//a)Reduce the Indegree of Vertex by 1

```
indegrees[mynode.vertex]--;
```

//b)If Indegree of that vertex

//becomes zero then add to Queue

```
if(indegrees[mynode.vertex]==0)  
myque.add(mynode.vertex);  
}
```

```
}
```

```
return topOrder;
```

```
}
```



```
public static void main(String[] args) {  
  
    MyDirectedGraph mygraph = new MyDirectedGraph(6);  
  
    mygraph.addEdge(0, 1,2);  
    mygraph.addEdge(0, 4,1);  
  
    mygraph.addEdge(1, 2,3);  
    mygraph.addEdge(4,2,2);  
  
    mygraph.addEdge(2, 3,6);  
    mygraph.addEdge(4, 5,4);  
    mygraph.addEdge(5, 3,1);  
  
    mygraph.printGraph();  
    ArrayList<Integer> topOrder = topologicalOrder(mygraph);  
    System.out.println(topOrder);  
}  
}
```



17.6.9. Shortest Path in DAG

Problem :

- ✓ Given **Directed Acyclic Graph**.
- ✓ You need to Find the Shortest from Source Vertex to all other Vertices.

Lab17.java

```
package com.myjlc.graphs.problems;

import java.util.*;
public class Lab17 {

    public static ArrayList<Integer> topologicalOrder(MyDirectedGraph mygraph)
    {
        ArrayList<Integer> topOrder = new
        ArrayList<Integer>(mygraph.vertexCount);

        // Step1: Find Indegrees of All Vertices
        int indegrees[] = new int[mygraph.vertexCount];

        for (int i = 0; i < indegrees.length; i++) {
            for (AdjacentNode mynode : mygraph.adjList.get(i)) {
                indegrees[mynode.vertex]++;
            }
        }

        // Step2: Create Queue
        Queue<Integer> myque = new LinkedList<Integer>();

        // Step3: Add Zero-Indegree Vertices to Queue
        for (int i = 0; i < indegrees.length; i++) {
            if (indegrees[i] == 0)
                myque.add(i);
        }
    }
}
```



```
// 4. When Queue is Not Empty
while (!myqueue.isEmpty()) {

    // 4A. Get Vertex from Queue
    int currVertex = myqueue.poll();
    topOrder.add(currVertex);

    // 4B. For each Adjacent Vertex
    for (AdjacentNode mynode : mygraph.adjList.get(currVertex)) {

        // a) Reduce the Indegree of Vertex by 1
        indegrees[mynode.vertex]--;

        // b) If Indegree of that vertex becomes zero then add to Queue
        if (indegrees[mynode.vertex] == 0)
            myqueue.add(mynode.vertex);
    }
}
return topOrder;
}

public static int[] shortestPaths(MyDirectedGraph mygraph, int sourceVertex)
{

    // Step1: Define Distance Array with Max
    int[] dist = new int[mygraph.vertexCount];

    for (int i = 0; i < dist.length; i++) {
        dist[i] = Integer.MAX_VALUE;
    }

    // Step2: Initialize Source Dist to Zero
    dist[sourceVertex] = 0;

    // Step3: Find Topological Order of Graph
    ArrayList<Integer> topOrder = topologicalOrder(mygraph);
```



```
// Step4: For each Vertex in Topological Order
for (int currVertex : topOrder) {

    // 4A. For each Adjacent Vertex
    for (AdjacentNode mynode : mygraph.adjList.get(currVertex)) {
        int adjVertex = mynode.vertex;
        int weight = mynode.weight;
        if (dist[adjVertex] > dist[currVertex] + weight) {
            dist[adjVertex] = dist[currVertex] + weight;
        }
    }
}
return dist;
}

public static void main(String[] args) {
    MyDirectedGraph mygraph = new MyDirectedGraph(6);
    mygraph.addEdge(0, 1, 2);
    mygraph.addEdge(0, 4, 1);
    mygraph.addEdge(1, 2, 3);
    mygraph.addEdge(4, 2, 2);
    mygraph.addEdge(2, 3, 6);
    mygraph.addEdge(4, 5, 4);
    mygraph.addEdge(5, 3, 1);

    int[] dist = shortestPaths(mygraph, 0);

    for (int i = 0; i < dist.length; i++)
        System.out.println(dist[i]);
}
}
```



17.6.10. Dijkstra's Algorithm

Problem :

- ✓ Given Directed or Undirected Graph with Weights on Edges.
- ✓ You need to Find the Shortest from Source Vertex to all other Vertices.
- ✓ **Dijkstra's Algorithm**

AdjacentNode.java

```
package com.myjlc.graphs.problems;
```

```
public class AdjacentNode {  
    int vertex;  
    int weight;  
  
    public AdjacentNode() {  
    }  
  
    public AdjacentNode(int vertex, int weight) {  
        super();  
        this.vertex = vertex;  
        this.weight = weight;  
    }  
  
    @Override  
    public String toString() {  
        return "[vertex=" + vertex + ", weight=" + weight + "];"  
    }  
}
```



MyGraph.java

```
package com.myjlc.graphs.problems;

import java.util.ArrayList;
import java.util.LinkedList;

class MyGraph {

    int vertexCount;
    ArrayList<LinkedList<AdjacentNode>> adjList;

    public MyGraph(int vertexCount) {

        this.vertexCount = vertexCount;
        this.adjList = new ArrayList<LinkedList<AdjacentNode>>(vertexCount);

        for (int i = 0; i < vertexCount; i++) {
            this.adjList.add(new LinkedList<AdjacentNode>());
        }

        public void addEdge(int vertex1, int vertex2,int weight) {
            AdjacentNode mynode = new AdjacentNode(vertex2,weight);
            adjList.get(vertex1).add(mynode);
        }

        public void printGraph() {
            for (int i = 0; i < vertexCount; i++) {
                System.out.print("Vertex : " + i + "=> \t");
                for (int j = 0; j < adjList.get(i).size(); j++) {
                    System.out.print(adjList.get(i).get(j) + "\t");
                }
                System.out.println("");
            }
        }
    }
}
```



Lab18.java

```
package com.myjlc.graphs.problems;

import java.util.PriorityQueue;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab18 {

    public static int[] dijkstrasPaths(MyGraph mygraph, int sourceVertex) {

        // Step1: Define Distance Array with Max
        int[] dist = new int[mygraph.vertexCount];

        for (int i = 0; i < dist.length; i++) {
            dist[i] = Integer.MAX_VALUE;
        }

        // Step2: Create PriorityQueue
        PriorityQueue<AdjacentNode> myque = new PriorityQueue<>((v1, v2) ->
            v1.weight - v2.weight);

        // Step3: Initialize Source Dist to Zero
        dist[sourceVertex] = 0;

        // Step4: Add SourceVertex Weight
        myque.add(new AdjacentNode(sourceVertex, 0));

        // Step5: For each Vertex in Queue
        while (!myque.isEmpty()) {

            //5A. Poll the vertex from Queue
            AdjacentNode vertex1 = myque.poll();
```



```
// 4A. For each Adjacent Vertex
for (AdjacentNode mynode : mygraph.adjList.get(vertex1.vertex)) {
    int vertex2 = mynode.vertex;
    int weight = mynode.weight;
    if (dist[vertex2] > dist[vertex1.vertex] + weight) {
        dist[vertex2] = dist[vertex1.vertex] + weight;
        myque.add(new AdjacentNode(vertex2,dist[vertex2]));
    }
}
return dist;
}

public static void main(String[] args) {
    /*
    MyGraph mygraph = new MyGraph(4);

    mygraph.addEdge(0, 1, 5);
    mygraph.addEdge(0, 2, 10);

    mygraph.addEdge(1, 0, 5);
    mygraph.addEdge(1, 2, 3);
    mygraph.addEdge(1, 3, 20);

    mygraph.addEdge(2, 0, 10);
    mygraph.addEdge(2, 1, 3);
    mygraph.addEdge(2, 3, 2);

    mygraph.addEdge(3, 1, 20);
    mygraph.addEdge(3, 2, 2);
    */
}
```



```
MyGraph mygraph = new MyGraph(4);

mygraph.addEdge(0, 1, -1);
mygraph.addEdge(0,2, 4);
mygraph.addEdge(1,2, -3);
mygraph.addEdge(1,3, -2);
mygraph.addEdge(2,3, 3);

int[] dist = dijkstrasPaths(mygraph, 0);

for (int i = 0; i < dist.length; i++)
    System.out.println(dist[i]);

}
}
```



17.6.11. Bellman-Ford Algorithm

Problem :

- ✓ Given Directed or Undirected Graph with Weights on Edges.
- ✓ You need to Find the Shortest from Source Vertex to all other Vertices.
- ✓ **Bellman-Ford Algorithm**

MyEdge.java

```
package com.myjlc.graphs.problems;
```

```
public class MyEdge {
    int sourceVertex;
    int destVertex;
    int weight;

    public MyEdge() {
    }

    public MyEdge(int sourceVertex,int destVertex,int weight) {
        super();
        this.sourceVertex = sourceVertex;
        this.destVertex = destVertex;
        this.weight = weight;
    }

    @Override
    public String toString() {
        return " [sourceVertex=" + sourceVertex + ", destVertex=" +
destVertex + ", weight=" + weight + "]";
    }
}
```



MyGraph.java

```
package com.myjlc.graphs.problems;

import java.util.ArrayList;

class MyGraph {

    int vertexCount;
    ArrayList<MyEdge> edgeList;

    public MyGraph(int vertexCount) {
        this.vertexCount = vertexCount;
        this.edgeList = new ArrayList<MyEdge>();
    }

    public void addEdge(int vertex1, int vertex2, int weight) {
        MyEdge myedge = new MyEdge(vertex1, vertex2, weight);
        edgeList.add(myedge);
    }

}
```

Lab19.java

```
package com.myjlc.graphs.problems;

import java.util.PriorityQueue;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab19 {
```



```
public static int[] bellmanFords(MyGraph mygraph, int sourceVertex) {
```

```
// Step1: Define Distance Array with Max
```

```
int[] dist = new int[mygraph.vertexCount];
```

```
for (int i = 0; i < dist.length; i++) {
```

```
dist[i] = Integer.MAX_VALUE;
```

```
}
```

```
// Step2: Initialize Source Dist to Zero
```

```
dist[sourceVertex] = 0;
```

```
// Step3: For each Vertex (v-1) times
```

```
for(int count=1;count<mygraph.vertexCount;count++) {
```

```
// 4A. For each Adjacent Vertex
```

```
for (MyEdge myedge : mygraph.edgeList) {
```

```
int vertex1 = myedge.sourceVertex;
```

```
int vertex2 = myedge.destVertex;
```

```
int weight = myedge.weight;
```

```
if (dist[vertex2] > dist[vertex1] + weight) {
```

```
dist[vertex2] = dist[vertex1] + weight;
```

```
}
```

```
}
```

```
}
```

```
return dist;
```

```
}
```



```
public static void main(String[] args) {
    /*
    MyGraph mygraph = new MyGraph(4);
    mygraph.addEdge(0, 1, 5);
    mygraph.addEdge(0, 2, 10);

    mygraph.addEdge(1, 0, 5);
    mygraph.addEdge(1, 2, 3);
    mygraph.addEdge(1, 3, 20);

    mygraph.addEdge(2, 0, 10);
    mygraph.addEdge(2, 1, 3);
    mygraph.addEdge(2, 3, 2);

    mygraph.addEdge(3, 1, 20);
    mygraph.addEdge(3, 2, 2);
    */

    MyGraph mygraph = new MyGraph(4);
    mygraph.addEdge(0, 1, 1);
    mygraph.addEdge(0,2, 4);
    mygraph.addEdge(1,2, -3);
    mygraph.addEdge(1,3, 2);
    mygraph.addEdge(2,3, 3);

    int[] dist = bellmanFords(mygraph, 0);

    for (int i = 0; i < dist.length; i++)
        System.out.println(dist[i]);
    }
}
```